
From Swagger to TypeScript

Designing APIs with clarity, power, and a single source of truth

Topics covered

- The Problem with OpenAPI / Swagger
- What is TypeScript?
- Code-First with TypeScript
- Core Syntax & Features
- The TypeScript Ecosystem
- Hands-On Walkthrough
- Spread, Composition & UUID
- Emitters, Linters & `@pattern`
- Alternative Solutions (Smithy...)

~30 min presentation

+ hands-on demo

Presenter: **radoslav.todorov@wsa.com**

The Reality of Writing OpenAPI by Hand

```
# openapi.yaml - a "simple" endpoint
paths:
  /pets/{id}:
    get:
      summary: Get a pet by ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: A pet object
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Pet'
        '404':
          description: Not found

components:
  schemas:
    Pet:
      type: object
      required: [id, name]
      properties:
        id:
          type: string
        name:
          type: string
```

Verbosity

20+ lines to describe one GET endpoint. Real APIs have hundreds.

Duplication

Every shared model must be repeated or referenced with fragile `$ref` strings.

No abstraction

No generics, no inheritance, no templates. Copy-paste is the "pattern".

Hard to review

Large YAML files are unreadable in PRs and nearly impossible to diff.

More Pain Points at Scale

Consistency

10 teams → 10 different naming conventions, error shapes, pagination styles. No enforced standard.

Multi-protocol

OpenAPI only covers REST. gRPC, AsyncAPI, JSON Schema need completely separate files.

Tooling drift

Generated clients get out of sync. Different validators disagree on the same spec. Breaking changes slip through.

No type safety

YAML/JSON is text. Typos in `$ref`, wrong types, missing fields — all caught only at runtime.

Version sprawl

OpenAPI 2 vs 3.0 vs 3.1 — incompatibilities everywhere. Swagger UI, ReDoc, Redocly all behave differently.

Scaling

Microsoft's Azure REST APIs: **thousands** of endpoints. Managing raw OpenAPI at that scale is untenable.



TypeSpec was born from exactly this pain — at Microsoft, managing the Azure API surface.



What is TypeSpec?

A language for describing APIs — not just
documenting them

TypeSpec at a Glance

TypeSpec is an **open-source, Microsoft-built language** for describing cloud service APIs and shapes — then *generating* everything from that single source.

- 🧠 Strongly-typed, compiler-verified API definition language
- ⚙️ Compiles to OpenAPI 3, JSON Schema, Protobuf, and more
- 🔧 Extensible via libraries and custom emitters
- 🏭 Used internally at Microsoft to define the entire Azure API surface
- 🌐 Open source at github.com/microsoft/typespec

typespec.io

The Core Idea

Write your API once in TypeSpec.

Let the compiler produce OpenAPI, JSON Schema, docs, and client SDKs.

Inspired by TypeScript

Familiar syntax. If you know TypeScript, TypeSpec feels immediately natural.

Not a runtime framework

TypeSpec is a *design-time* tool. It doesn't run your API — it defines its contract.

The "Code-First" Approach with TypeScript

Traditional approaches to API design:

Design-First (old style)

Write YAML/JSON OpenAPI spec by hand → implement server → hope they stay in sync.

Code-First (old style)

Write server code → annotate with attributes → generate OpenAPI. Spec is a by-product, often imperfect.

TypeSpec Code-First

Write **TypeSpec** — a real programming language. Compile → get perfect, consistent OpenAPI (and more). The spec *is* the source of truth.

Why Code-First with TypeScript wins

- Refactor-safe: rename a model, everywhere updates
- IDE support: autocomplete, go-to-definition, error squiggles
- Reuse via imports, templates, libraries
- Version control friendly: small, readable diffs
- Enforces API style through shared libraries (like linting)
- One input → multiple outputs (OpenAPI, Protobuf, docs...)



Core Syntax Features

TypeSpec vs OpenAPI — side by side

Models → OpenAPI Schemas

TypeSpec

```
model Pet {  
  id:   string;  
  name: string;  
  age?: uint8 = 0;  
  kind: "cat" | "dog" | "bird";  
}  
  
model Dog extends Pet {  
  breed: string;  
}
```

Equivalent OpenAPI 3

```
components:  
  schemas:  
    Pet:  
      type: object  
      required: [id, name, kind]  
      properties:  
        id: { type: string }  
        name: { type: string }  
        age:  
          type: integer  
          format: uint8  
          default: 0  
        kind:  
          type: string  
          enum: [cat, dog, bird]  
    Dog:  
      allOf:  
        - $ref: '#/components/schemas/Pet'  
        - type: object  
          required: [breed]  
          properties:  
            breed: { type: string }
```

TypeSpec: **8 lines** | OpenAPI YAML: **22 lines** — and TypeSpec is statically checked.

Operations → OpenAPI Paths

TypeSpec

```
import "@typespec/http";
using TypeSpec.Http;

@route("/pets")
namespace Pets {

  @get
  op listPets(): Pet[];

  @post
  op createPet(@body pet: Pet): Pet;

  @get @route("/{id}")
  op getPet(@path id: string):
    Pet | NotFoundError;

  @delete @route("/{id}")
  op deletePet(@path id: string): void;
}
```

Generated OpenAPI (abbreviated)

```
paths:
  /pets:
    get:
      operationId: listPets
      responses:
        '200':
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Pet'
    post:
      operationId: createPet
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Pet'
      responses:
        '200': ...
  /pets/{id}:
    get: ...
    delete: ...
```

Decorators — Metadata as First-Class Citizens

Decorators attach HTTP, validation, and documentation metadata directly to types.

```
@doc("A registered pet")
model Pet {
  @doc("Unique identifier")
  @key
  id: string;

  @minLength(1) @maxLength(50)
  name: string;

  @minValue(0) @maxValue(30)
  age?: uint8;
}
```

Common built-in decorators

Decorator	Effect
@doc	description in generated spec
@route	URL path segment
@get @post @put @delete @patch	HTTP verb
@path @query @header @body @cookie	parameter location
@minLength @maxLength	string constraints
@minValue @maxValue	numeric constraints
@format	e.g. "uuid", "date-time"
@pattern	regex constraint on string values
@key	marks the resource identifier

Interfaces & Namespaces — Organise at Scale

Grouping operations with `interface`

```
@route("/pets")
interface PetsService {
  @get
  op list(): Pet[];

  @post
  op create(@body b: CreatePetRequest):
    Pet;
}
```

```
// Separate namespace for admin routes
@route("/admin")
namespace Admin {
  @get @route("/pets")
  op listAll(): Pet[];
}
```

namespace

Logical grouping. Translates to a path prefix. Nestable. Mirrors how large API surfaces are split into domains.

interface

Groups a set of operations that share behaviour or a resource. Can be extended or referenced from templates — just like an interface in OO languages.

Combining both

Real APIs: namespace per domain (Users, Orders, Products), interface per resource within the domain.

Templates — Generics for APIs

Define once, use for every resource

```
// Generic paged response
model Page<T> {
    items: T[];
    nextLink?: string;
    totalCount: int32;
}

// Generic CRUD interface
interface CrudOps<TResource, TCreate> {
    @get    op list(): Page<TResource>;
    @post   op create(@body b: TCreate): TResource;
    @get    @route("/{id}")
            op read(@path id: string): TResource;
}

// Instantiate for concrete resources
@route("/pets")
interface Pets extends CrudOps<Pet, CreatePet> {}
```

Zero duplication

10 resources → 10 one-liner interfaces. All pagination, error handling, and CRUD shapes defined *once*.

Compiler-checked

If you forget a field in a model passed to the template, you get a compile error — not a broken spec.

OpenAPI has no equivalent

Raw OpenAPI forces copy-paste for every resource. TypeSpec's templates are a *transformative* advantage.

Union Types & Structured Error Responses

```
// Standard error envelope
model ApiError {
  code:    string;
  message: string;
}

@error
model NotFound    is ApiError;
@error
model Unauthorized is ApiError;
@error
model Conflict     is ApiError;

// Return type is a union
op getPet(@path id: string):
  Pet | NotFound | Unauthorized;
```

Union types

Operations can return multiple response types. TypeSpec maps each to the appropriate HTTP status code automatically when combined with `@statusCode`.

@error decorator

Marks a model as an error response. The emitter knows to map it to 4xx/5xx status codes in OpenAPI.

Consistent errors, always

Define `ApiError` once in a shared library. Every team, every service uses the same error shape — enforced by the compiler.

Scalars, Enums & Type Aliases

Custom scalars (branded types)

```
// Extend built-in scalars
@format("uuid")
scalar Uuid extends String;

@minLength(1) @maxLength(255)
scalar NonEmptyString extends String;

// Regex-validated scalar
@pattern("^[\\w.+~]+@[\\w-]+\\.([\\w.]+)$")
scalar EmailAddress extends String;

// Enums
enum PetKind {
    Cat: "cat",
    Dog: "dog",
    Bird: "bird",
}

model Pet {
    id: Uuid;
    name: NonEmptyString;
```

Scalars

Create domain-specific types carrying validation rules — length, range, `@format("uuid")`, and `@pattern("regex")`. Use them everywhere for compile-time safety; impossible to accidentally pass a bare string.

Enums

Named, strongly-typed enumerations. Compile to OpenAPI enum arrays. IDE gives you autocomplete on enum values.

vs OpenAPI

OpenAPI has no custom scalars. You'd have to copy `format: "uuid"` and `minLength: 1` onto every single field. Forever.

TypeSpec vs OpenAPI — Feature Comparison

Feature	Raw OpenAPI YAML	TypeSpec
Strongly typed	✗ Text/YAML	✓ Compiler-checked
Generics / Templates	✗ None	✓ Full generic syntax
Inheritance	⚠ allOf workaround	✓ extends, is
Custom scalar types	✗ Copy-paste constraints	✓ scalar keyword
IDE support	⚠ Schema-based only	✓ Full LSP (VS Code / VS)
Multi-protocol output	✗ REST only	✓ OpenAPI, Protobuf, JSON Schema...
Reusable libraries	✗ Copy files	✓ npm packages
Readability at scale	✗ Thousands of lines YAML	✓ Concise, modular
Versioning support	⚠ Manual	✓ @typespec/versioning library



The TypeScript Ecosystem

Libraries, emitters, tooling, and integrations

Official Libraries

@typespec/http

HTTP routing decorators: `@route`, `@get`, `@post`, `@body`, `@path`, `@query`, `@header`

@typespec/rest

REST resource patterns: `Resource`, `CollectionWithNextLink`. Builds on `@typespec/http`.

@typespec/openapi3

Emitter that produces OpenAPI 3.0/3.1 YAML or JSON from your TypeScript.

@typespec/versioning

Declare API versions. Mark properties as added/removed in specific versions. Generates per-version OpenAPI.

@typespec/json-schema

Emit pure JSON Schema from TypeScript models — useful for message validation, config schemas.

@typespec/protobuf

Emit `.proto` files for gRPC services from the same TypeScript source.

@azure-tools/typespec-azure-core

Azure-specific patterns. Used internally at Microsoft to define the entire Azure REST API surface.

@typespec/compiler

The core compiler. Provides the LSP server, CLI (`tsp`), formatter, and extension APIs.

Community

Growing ecosystem of community emitters, linters, and style libraries published on npm.

Developer Tooling

VS Code Extension

Full Language Server Protocol support: syntax highlighting, autocomplete, go-to-definition, inline errors, hover docs, formatter.

Visual Studio Extension

Same LSP features for the .NET ecosystem.

TypeSpec Playground

Online editor at typespec.io/playground — write TypeSpec, see OpenAPI output instantly. No install needed. Great for demos.

CLI — tsp

```
# Scaffold a new project
tsp init

# Compile & emit
tsp compile .

# Watch mode
tsp compile . --watch

# Format all .tsp files
tsp format **/*.tsp
```

tspconfig.yaml

Controls which emitters run, output paths, compiler options — per-project, version-controlled.

API Versioning — Built-In

```
import "@typespec/versioning";
using TypeSpec.Versioning;

@versioned(Versions)
namespace PetStore {

    enum Versions { v1, v2 }

    model Pet {
        id: string;
        name: string;

        // Added in v2 – invisible in v1 spec
        @added(Versions.v2)
        microchipId?: string;
    }

    // Whole operation added in v2
    @added(Versions.v2)
    @get @route("/pets/microchip/{id}")
    op findByMicrochip(@path id: string): Pet;
}
```

One source — multiple versions

The emitter generates a *separate* OpenAPI file per version. The diff between versions is tracked in the TypeSpec source.

Available annotations

- @added(version) — when a field/op appeared
- @removed(version) — when deprecated/removed
- @renamedFrom(version, oldName)
- @madeOptional(version)

vs OpenAPI

Raw OpenAPI: maintain two completely separate, manually synced YAML files. TypeSpec: single file, compiler does the rest.

TypeSpec in the Wider Stack



Input

.tsp files
Shared libraries
tspconfig.yaml



TypeSpec Compiler

Type checking
Decorator resolution
Emitter pipeline



Outputs

OpenAPI 3.x YAML
JSON Schema
.proto (gRPC)
Docs, SDK stubs

Downstream from OpenAPI

Once you have a clean, TypeSpec-generated OpenAPI spec you can use the full existing ecosystem: **Swagger UI**, **Redocly**, **AutoRest**, **Kiota**, **NSwag**, **OpenAPI Generator** for client SDKs in any language.

Azure & Microsoft adoption

TypeSpec is the *canonical* way to define Azure REST APIs. The Azure SDK team uses it to generate SDKs for .NET, Java, Python, JavaScript, Go automatically from the same TypeSpec source.

Key Takeaways

The problem

Raw OpenAPI YAML is verbose, fragile, doesn't scale, and forces copy-paste patterns across teams.

What TypeSpec is

A strongly-typed, compiler-checked language that generates OpenAPI and more from a clean, concise source.

Code-first done right

The TypeSpec file is the source of truth — not the generated YAML. Refactoring, versioning, and consistency become trivial.

Killer features

Generics/templates, custom scalars, decorators, built-in versioning, multi-protocol output.

Ecosystem

Official emitters for OpenAPI, JSON Schema, Protobuf. VS Code extension with full IDE support. Growing npm ecosystem.

Start today

```
npm install -g @typespec/compiler  
tsp init    # pick "Generic REST API"  
tsp compile .
```



Hands-On Demo

Building a Pet Store API from scratch

Live coding in VS Code



HANDS-ON — STEP 1

Project Setup

- 1 Install the TypeScript compiler globally:
`npm install -g @typespec/compiler`
- 2 Install the **TypeSpec for VS Code** extension from the marketplace.
- 3 Create a new project folder and run:
`tsp init`
- 4 Select the **Generic REST API** template.
Make sure `@typespec/openapi3` is selected.
- 5 Open the folder in VS Code.
Show the generated `main.tsp` and `tspconfig.yaml`.

Generated `tspconfig.yaml`

```
emit:
  - "@typespec/openapi3"

options:
  "@typespec/openapi3":
    emitter-output-dir: "{output-dir}/schema"
    openapi-versions:
      - 3.1.0
```

Compile and inspect output

```
tsp compile .
# → tsp-output/schema/openapi.yaml
```



HANDS-ON — STEP 2

Define Your First Model

Replace `main.tsp` with:

```
import "@typespec/http";
using Http;

@service({ title: "Pet Store API" })
namespace PetStore;

model Pet {
  @doc("Unique pet identifier")
  id: string;

  @minLength(1)
  name: string;

  kind: "cat" | "dog" | "bird";
  age?: uint8;
}

model CreatePetRequest {
  @minLength(1)
  name: string;

  kind: "cat" | "dog" | "bird";
```

Things to show

- Autocomplete on types (`uint8` etc.)
- Hover over `@minLength` to see docs
- Introduce a type error — see red squiggle

Compile & inspect

Run `tsp compile .` and open the generated `openapi.yaml`.
Show the `components/schemas` section.

Tip

Use `tsp compile . --watch` for live output while editing.



HANDS-ON — STEP 3

Add CRUD Operations

Add below the models in `main.tsp`:

```
@route("/pets")
interface Pets {

  @get
  op listPets(): Pet[];

  @post
  op createPet(
    @body body: CreatePetRequest
  ): Pet;

  @get @route("/{id}")
  op getPet(@path id: string): Pet;

  @put @route("/{id}")
  op updatePet(
    @path id: string,
    @body body: CreatePetRequest
  ): Pet;

  @delete @route("/{id}")
  op deletePet(@path id: string): void;
```

What to show

- Recompile and show generated paths in OpenAPI
- Count lines: ~30 TypeSpec vs ~120 OpenAPI YAML
- Open typespec.io/playground — paste the code, see output live

Discussion point

What if you need to add `@query` filter params to `listPets`? Show how simple it is vs editing raw YAML.



HANDS-ON — STEP 4

Add Error Handling

Add error models & update operations

```
@error
model NotFoundError {
  @statusCode code: 404;
  message: string;
}

@error
model ValidationError {
  @statusCode code: 400;
  message: string;
  fields: string[];
}

// Update getPet to return union
@get @route("/{id}")
op getPet(@path id: string):
  Pet | NotFoundError;

// Update createPet
@post
op createPet(@body b: CreatePetRequest):
  Pet | ValidationError;
```

Show in generated OpenAPI

The union types map to separate **responses** entries with the correct HTTP status codes — automatically.

Consistency win

Define **NotFoundError** once in a shared file. Import it everywhere. All 404s look the same across all endpoints.

Next step idea

Extract models into a `models.tsp` file and use `import` `"./models.tsp"` to show modular project structure.



HANDS-ON — STEP 5

Templates & Reuse (Bonus)

Introduce a reusable paged response

```
// shared/pagination.tsp
model Page<T> {
  items: T[];
  totalCount: int32;
  nextLink?: string;
}

// Update listPets in main.tsp
import "./shared/pagination.tsp";

@get
op listPets(
  @query skip?: int32,
  @query top?: int32
): Page<Pet>;
```

The pay-off

Add a second resource (e.g. `Owner`). The same `Page<T>` works instantly — zero duplication, compiler-verified.

Show the final OpenAPI

Open the generated YAML. Highlight that a clean, production-grade spec was created with ~60 lines of readable TypeSpec.

Where to go next

- Step 6 → Spread / Composition & UUID scalars
- Step 7 → Emitters & Linters
- `tsp init` with Azure template



HANDS-ON — STEP 6

Spread / Composition & UUID

Compose models with the spread operator

```
// Custom scalar - UUID identifier
@format("uuid")
scalar PetId extends string;

// Length + regex constraint
@minLength(1) @maxLength(100)
@pattern("^[A-Za-z ]+$")
scalar PetName extends string;

// Shared base model
model PetBase {
  name: PetName;
  kind: "cat" | "dog" | "bird";
  age?: uint8;
}

// Spread PetBase - add id on top
model Pet {
  id: PetId;
  ... PetBase;
}
```

Spread operator ...

Inlines all properties of the target model — exactly like TypeScript object spread. Ideal for the Create / Update / Read pattern where models differ only by presence of `id`.

UUID scalar

`@format("uuid")` emits `format: uuid` in OpenAPI. A branded type — prevents mixing up IDs with plain strings across operations at compile time.

@pattern

Attaches a regex constraint. Emits `pattern: " ... "` in JSON Schema / OpenAPI 3. Great for names, slugs, phone numbers, emails — anything with a fixed format.

Zero duplication

Add a field to `PetBase` once — it propagates automatically to both



HANDS-ON — STEP 7

Emitters & Linters

Step 1 — install packages (@typespec/versioning is a required peer dep)

```
npm install @azure-tools/typespec-azure-core
npm install @typespec/versioning
```

Step 2 — add both rulesets to tspconfig.yaml

```
linter:
  extends:
    - "@typespec/http/all"          # HTTP-specific rules
    - "@azure-tools/typespec-azure-core/all" # Azure API Guidelines
```

Step 3 — compile and see real warnings (44 on our Pet Store)

```
main.tsp:8 - warning documentation-required:
  Model "Pet" must have a documentation comment.
main.tsp:28 - warning documentation-required:
  Operation "listPets" must have a @doc decorator.
main.tsp:4 - warning require-versioned:
  Namespace "PetStore" should use the @versioned decorator.
main.tsp:17 - warning no-unnamed-union:
  "cat"|"dog"|"bird" should be a named enum declaration.
```

Step 4 — fix or suppress inline in .tsp (not in tspconfig!)

```
// ✅ Fix: add doc comment
@doc("Returns all pets, paginated")
@get
```

Compiler vs Linter

Compiler — type errors. The spec won't generate at all.

Linters — design quality. The spec generates but would be incomplete, ambiguous, or violate API guidelines. It's the difference between *"it works"* and *"it's correct"*.

Real warnings from azure-core/all

Rule	What it flags
documentation-required	Missing @doc on any model or op
require-versioned	No @versioned on namespace
auth-required	No @useAuth on service
no-unnamed-union	Inline union should be a named enum
no-format	@format("uuid") — use Azure scalars



rules: in tspconfig.yaml does NOT work

Per-rule overrides are **not supported** in tspconfig.yaml — the

HANDS-ON — STEP 8 API Versioning

Install, then annotate your namespace & model

```
npm install @typespec/versioning
```

```
import "@typespec/versioning";
using TypeSpec.Versioning;

// 1. Declare the supported versions
@versioned(Versions)
@service({ title: "Pet Store API" })
namespace PetStore {

    enum Versions { v1, v2 }

    model Pet {
        id: PetId;
        ... PetBase;

        // 2. Field added only in v2 - invisible in v1 spec
        @added(Versions.v2)
        microchipId?: string;
    }

    // 3. Whole operation added in v2
```

One source → two specs

The compiler generates a *separate* `openapi.yaml` per version. The diff between versions lives in your TypeSpec source — tracked in Git, reviewed in PRs.

```
tsp-output/
  schema/
    openapi.v1.yaml
    openapi.v2.yaml
```

Version annotations

Decorator	Meaning
@versioned(V)	Marks a namespace as versioned
@added(V.v2)	Appeared in v2, absent in v1
@removed(V.v2)	Gone from v2 onward
@renamedFrom(V.v2, "old")	Renamed between versions
@madeOptional(V.v2)	Required in v1, optional in v2

vs raw OpenAPI

Alternative Solutions

Tool	Origin	Primary Output	Strengths	Limitations
TypeSpec	Microsoft	OpenAPI, JSON Schema, Protobuf	Generics, decorators, multi-protocol, TypeScript syntax	Younger ecosystem, MS-focused docs
Smithy	AWS	OpenAPI, AWS SDK, Smithy IDL	Mature, AWS SDK codegen, rich trait system	Java-centric tooling, steeper learning curve
Raw OpenAPI 3.1	OpenAPI Initiative	OpenAPI (is the output)	Universal standard, massive tooling ecosystem	Verbose YAML, no generics, fragile \$ref
Protobuf / gRPC	Google	.proto, gRPC stubs	High performance, polyglot codegen, compact wire format	Binary protocol, not REST/browser-friendly
GraphQL SDL	Meta	GraphQL schema	Self-describing, flexible client-driven queries	Different paradigm, not REST/HTTP native
RAML	MuleSoft	OpenAPI (via converter)	Clean YAML DSL, MuleSoft integration	Declining adoption, limited modern tooling

TypeSpec vs Smithy

Both solve the same core problem from the two largest cloud API producers. TypeSpec has TypeScript-like syntax and broader protocol support out of the box. Smithy has a more mature AWS SDK generation story. Pick TypeSpec for REST-first or multi-protocol needs; Smithy if you're building deep inside the AWS SDK

The broader trend

The industry is converging on IDL-based API design. Raw OpenAPI YAML is increasingly treated as a *compilation target*, not a source of truth. TypeSpec and Smithy both validate this direction — from the two companies responsible for the world's largest public API surfaces.

Resources & Q&A

 **Official site**

`typespec.io`

 **Playground**

`typespec.io/playground`

 **GitHub**

`github.com/microsoft/typespec`

 **Discord**

`aka.ms/typespec/discord`

 **Docs**

`typespec.io/docs`

Thank you!

TypeSpec gives your team a **single source of truth** for API contracts — expressive, refactor-safe, and capable of targeting multiple protocols from one definition.

Questions?